大连东软信息学院
Dalian Neusoft University of Information

课堂实验报告

表 1　实验基本信息

姓名：庄锦泰　　学号：25003130516			
实验名称	农作物病虫害识别与分类	指导教师	
实验地点	A2-205a	实验日期	2026-11-04

一、实验数据

实验环境：Windows 11 + NVIDIA GeForce RTX 5060 Laptop GPU，Python 3.13，PyTorch 2.11（CUDA 12.8），输入尺寸 224×224，batch=32，优化器 Adam（lr=1e-4）。

数据集 Pests_dataset 含苹果、葡萄两类作物 7 类叶片图像 4173 张，另有 1213 张无标签测试图用于推理演示。类别为苹果黑星病（450）、黑腐病（444）、雪松锈病（198）、健康（1183），葡萄黑腐病（846）、健康（299）、褐斑病（753），均为 256×256 RGB JPG，最多类与最少类约 6:1，各类样例图像如表 2 所示。官方 val 的 466 张不参与训练；为统一训练/验证比例，从 Train 中按类别分层重切 20% 作验证集（836 张），余 3337 张用于训练，验证集不参与训练，仅用于模型选择与精度评估。

表 2　七类叶片样例图像


苹果黑星病


苹果黑腐病


苹果雪松锈病


苹果健康


葡萄黑腐病


葡萄健康


葡萄褐斑病

二、实验过程

1. 基础实验

（1）自定义卷积神经网络 SimpleCNN（自定义 forward）

网络由 3 组「卷积→ReLU→最大池化」与两层全连接构成：输入 224×224×3，经 32/64/128 通道卷积逐级提特征并下采样至 28×28，Flatten 后接 256 维全连接与 Dropout(0.5)，最后由 7 维输出经 Softmax 得到类别概率。

SimpleCNN 模型结构（print(model) 输出）

```
SimpleCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU()
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=100352, out_features=256, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=256, out_features=7, bias=True)
  )
)
```

图 1　SimpleCNN 模型结构

models.py - 自定义卷积神经网络 SimpleCNN（自定义 forward）

```python
class SimpleCNN(nn.Module):
    """3 组 (Conv3x3→ReLU→MaxPool) + 两层全连接，结构贴近指导书示意"""

    def __init__(self, num_classes=7):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 224→112
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 112→56
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 56→28
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 28 * 28, 256), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

图 2　SimpleCNN 代码（自定义 forward）

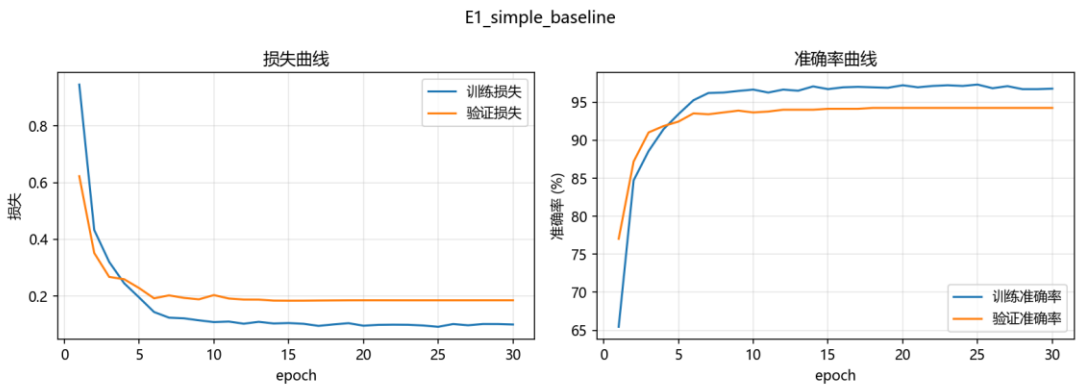


图 3　SimpleCNN 训练曲线

E1 SimpleCNN 训练结果（含最佳验证准确率）

```
epoch  25/30 | lr 1.00e-09 | train loss 0.0918 acc 97.33% | val loss 0.1851 acc 94.26%
epoch  26/30 | lr 1.00e-09 | train loss 0.1014 acc 96.85% | val loss 0.1851 acc 94.26%
epoch  27/30 | lr 1.00e-09 | train loss 0.0970 acc 97.12% | val loss 0.1851 acc 94.26%
epoch  28/30 | lr 1.00e-09 | train loss 0.1015 acc 96.73% | val loss 0.1851 acc 94.26%
epoch  29/30 | lr 1.00e-09 | train loss 0.1014 acc 96.73% | val loss 0.1851 acc 94.26%
epoch  30/30 | lr 1.00e-10 | train loss 0.0998 acc 96.79% | val loss 0.1851 acc 94.26%

[完成] 最佳验证准确率 94.26% (epoch 18)，用时 4.6 分钟
```

图 4　SimpleCNN 训练结果（含准确率）

（2）调用经典 CNN 模型（ResNet18，ImageNet 预训练）

调用 torchvision 自带的经典 CNN 模型 ResNet18，加载 ImageNet 预训练权重做迁移学习，仅把最后 fc 层替换为 7 类输出后整体微调，与自定义模型采用完全相同的训练配置。在该 836 张验证集上，第 1 轮即达 100.00%，30 轮内各轮均不低于 99.88%，末轮仍为 100.00%。四组实验的配置与验证准确率汇总见表 3。

models.py - 调用 torchvision 经典 CNN（ResNet18 预训练）

```
def _tv_resnet18(num_classes, pretrained):
    w = torchvision.models.ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
    m = torchvision.models.resnet18(weights=w)
    m.fc = nn.Linear(m.fc.in_features, num_classes)
    return m


def _tv_vgg16(num_classes, pretrained):
BUILDERS = {
    "simple": lambda n, p: SimpleCNN(n),
    "deep": lambda n, p: DeepCNN(n),
    "residual": lambda n, p: ResidualCNN(n),
    "dwsep": lambda n, p: DWSepCNN(n),
    "se_residual": lambda n, p: SEResidualCNN(n),
    "resnet18": _tv_resnet18,
    "vgg16": _tv_vgg16,
    "mobilenet_v2": _tv_mobilenet_v2,
}


def build_model(name, num_classes=7, pretrained=False):
```

图 5　经典 CNN 调用代码

ResNet18 模型结构（torchvision，替换 fc 为 7 类输出）

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
    ... （中间 layer2/layer3/layer4 略）...
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=7, bias=True)
)
```

图 6　ResNet18 模型结构

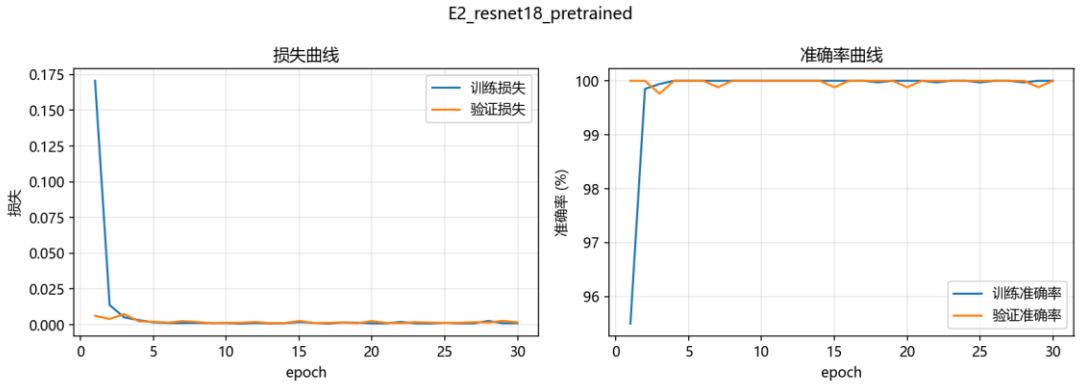


图 7　ResNet18（预训练）训练曲线

E2 ResNet18(预训练) 训练结果（含最佳验证准确率）

```
epoch  25/30 | lr 1.00e-09 | train loss 0.0013 acc 99.97% | val loss 0.0012 acc 100.00%
epoch  26/30 | lr 1.00e-09 | train loss 0.0010 acc 100.00% | val loss 0.0013 acc 100.00%
epoch  27/30 | lr 1.00e-09 | train loss 0.0010 acc 100.00% | val loss 0.0018 acc 100.00%
epoch  28/30 | lr 1.00e-09 | train loss 0.0026 acc 99.97% | val loss 0.0015 acc 100.00%
epoch  29/30 | lr 1.00e-09 | train loss 0.0010 acc 100.00% | val loss 0.0027 acc 99.88%
epoch  30/30 | lr 1.00e-10 | train loss 0.0011 acc 100.00% | val loss 0.0016 acc 100.00%

[完成] 最佳验证准确率 100.00% (epoch 1)，用时 5.7 分钟
```

图 8　ResNet18（预训练）训练结果（含准确率）

2.　创新实验

优化方法 1：改进网络模型（残差连接）

基线 SimpleCNN 的全连接层参数臃肿（25.8M）且深层特征提取不足。将网络改为手写残差结构 ResidualCNN：3 个 stage（64/128/256 通道），每个 stage 两个 BasicBlock（两个 3×3 卷积 + shortcut 跳连），末端用全局平均池化替代大规模全连接（参数量降为 2.8M）。训练配置与基线完全一致，验证准确率由 94.26% 提升至 97.97%（+3.71 个百分点）。

models.py - BasicBlock（含 use_se 分支）与 SEBlock 完整实现

```
class BasicBlock(nn.Module):
    """两个 3x3 卷积 + shortcut；形状变化时 shortcut 用 1x1 卷积投影"""

    def __init__(self, in_ch, out_ch, stride=1, use_se=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)
        self.se = SEBlock(out_ch) if use_se else nn.Identity()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_ch),
            )
        else:
            self.shortcut = nn.Identity()

    def forward(self, x):
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = self.se(out)
        out = out + self.shortcut(x)  # 残差连接
        return self.relu(out)
class SEBlock(nn.Module):
    """Squeeze-and-Excitation：全局池化→FC 降维→ReLU→FC 升维→sigmoid 门控"""

    def __init__(self, channels, reduction=16):
        super().__init__()
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(channels, channels // reduction), nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels), nn.Sigmoid(),
        )

    def forward(self, x):
        b, c, _, _ = x.shape
        w = self.pool(x).view(b, c)
        w = self.fc(w).view(b, c, 1, 1)
        return x * w
```

图 9　BasicBlock 与 SEBlock 完整实现（右为优化 2 使用的 SE 变体）

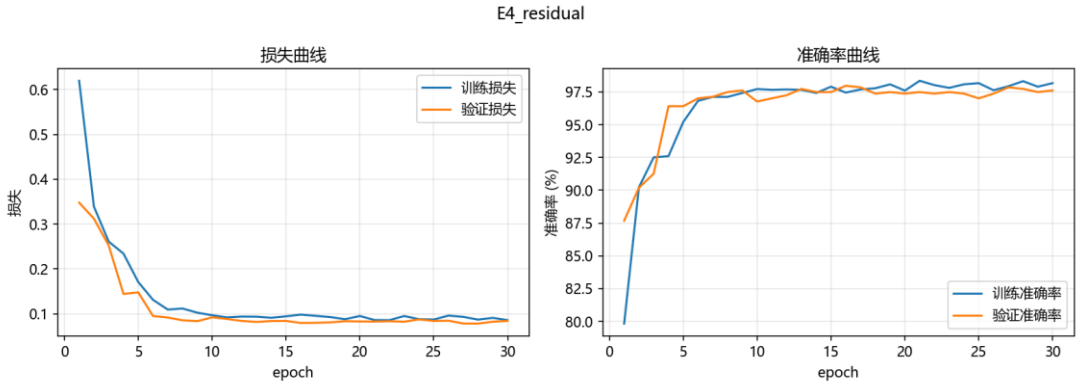


图 10　优化方法 1 训练曲线

```
E4 ResidualCNN(残差自建网络) 训练结果（含最佳验证准确率）
epoch  25/30 | lr 1.00e-09 | train loss 0.0872 acc 98.17% | val loss 0.0841 acc 97.01%
epoch  26/30 | lr 1.00e-09 | train loss 0.0961 acc 97.63% | val loss 0.0846 acc 97.37%
epoch  27/30 | lr 1.00e-09 | train loss 0.0932 acc 97.93% | val loss 0.0783 acc 97.85%
epoch  28/30 | lr 1.00e-09 | train loss 0.0871 acc 98.32% | val loss 0.0782 acc 97.73%
epoch  29/30 | lr 1.00e-09 | train loss 0.0910 acc 97.90% | val loss 0.0825 acc 97.49%
epoch  30/30 | lr 1.00e-10 | train loss 0.0859 acc 98.17% | val loss 0.0843 acc 97.61%

[完成] 最佳验证准确率 97.97% (epoch 16)，用时 17.1 分钟
```

图 11　优化方法 1 训练结果（含准确率）

优化方法 2：优化训练超参数（余弦退火）

StepLR 每 5 轮把学习率乘 0.1，训练后期学习率过小、参数更新几乎停滞。本组调度与训练预算同时改变：训练由 30 轮加长到 40 轮，学习率由 StepLR 换为余弦退火全程平滑衰减；损失函数、数据增强与网络结构均不变（对照为同一 SE-Residual 网络沿用 30 轮 StepLR，97.85%）。验证准确率升至 99.16%，较对照约 +1.2 个百分点（best 对 best 为 11 张图，final 对 final 为 10 张图）。该值取自 best_val_acc，是在与报告同一份 836 张验证集上逐轮取的最大值，含约 1 张图的选择偏差（P2 best 99.16% / final 99.04%），收益方向可信，但不宜按 0.01 个百分点的精度解读。

```
engine.py / train.py - 余弦退火学习率调度
def build_scheduler(name, optimizer, epochs):
    if name == "step":     # 指导书式：每 5 个 epoch 学习率 ×0.1
        return optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)
    if name == "cosine":   # 余弦退火
        return optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
    return None
    optimizer = optim.Adam(model.parameters(), lr=args.lr,
                           weight_decay=args.weight_decay)
    scheduler = build_scheduler(args.scheduler, optimizer, args.epochs)
```

图 12　余弦退火学习率调度代码

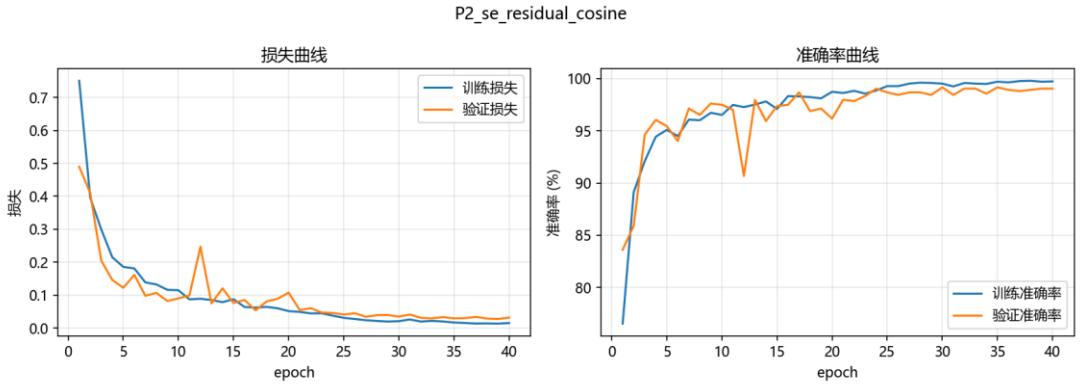


图 13　优化方法 2 训练曲线

```
P2 超参优化(余弦退火) 训练结果（含最佳验证准确率）
epoch  35/40 | lr 3.81e-06 | train loss 0.0174 acc 99.70% | val loss 0.0299 acc 99.16%
epoch  36/40 | lr 2.45e-06 | train loss 0.0159 acc 99.64% | val loss 0.0306 acc 98.92%
epoch  37/40 | lr 1.38e-06 | train loss 0.0143 acc 99.76% | val loss 0.0343 acc 98.80%
epoch  38/40 | lr 6.16e-07 | train loss 0.0146 acc 99.79% | val loss 0.0293 acc 98.92%
epoch  39/40 | lr 1.54e-07 | train loss 0.0141 acc 99.70% | val loss 0.0281 acc 99.04%
epoch  40/40 | lr 0.00e+00 | train loss 0.0157 acc 99.73% | val loss 0.0319 acc 99.04%

[完成] 最佳验证准确率 99.16% (epoch 30)，用时 25.0 分钟
```

图 14　优化方法 2 训练结果（含准确率）

附加实验 1：数据增强与训练预算的消融。对规整的实验室标准叶片图做强增强（随机裁剪/翻转/旋转/颜色抖动）并沿用 30 轮 StepLR 时，验证准确率仅 81.82%——StepLR 到第 30 轮已把学习率降到 1e-10、训练集准确率只有 79.35%，属训练预算不足导致的欠拟合。把轮数换成 40 轮余弦退火、其余不变（P5），同一强增强配置回升到 93.30%（+11.48 个百分点），说明低分主因是调度器而非增强本身；但相对无增强基线 94.26% 仍低 0.96 个百分点（两者调度器不同，仅作参考）。为把轻增强的独立贡献拆出来，在训练集缩减到 30%（1001 张）的条件下补齐四种组合：无增强+StepLR 89.00% → 轻增强+StepLR 86.24%（短预算下轻增强同样为负）→ 无增强+余弦 40 轮（P6）90.55%（仅换调度器与轮数就带来 +1.55）→ 轻增强+余弦 40 轮 92.82%。由此可分离出两个因素：余弦退火 + 40 轮本身贡献 1.55 个百分点，轻增强在预算匹配后再贡献 2.27 个百分点，两者叠加得到 92.82%。可见增强是否有效取决于训练预算：预算不足时增强放大欠拟合，预算匹配后轻增强才转为正收益；强增强即使补足预算（93.30%）仍略低于无增强基线。

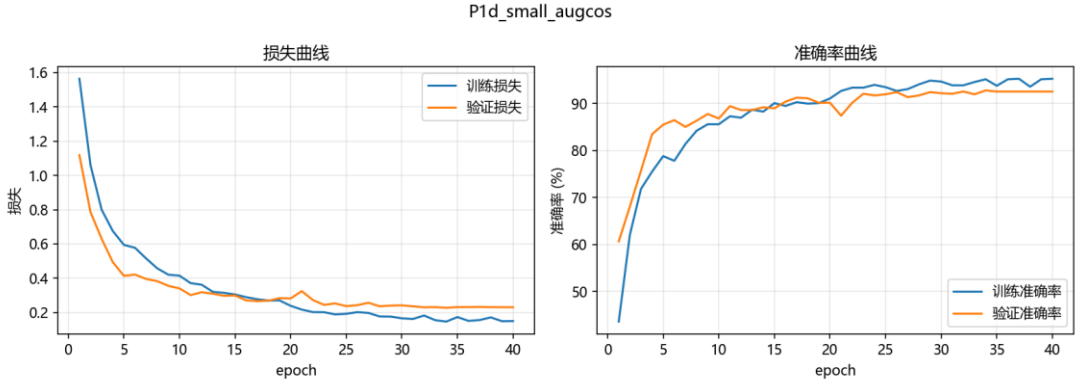


图 15　数据增强消融实验（30% 数据 + 轻增强 + 余弦退火）训练曲线

附加实验 2：组合实验。以 ResNet18 预训练模型叠加强数据增强、余弦退火与标签平滑，验证准确率达 100.00%；另有类别加权采样实验（98.80%，整体精度略降但改善小类召回）。各实验的验证准确率对比如下图。

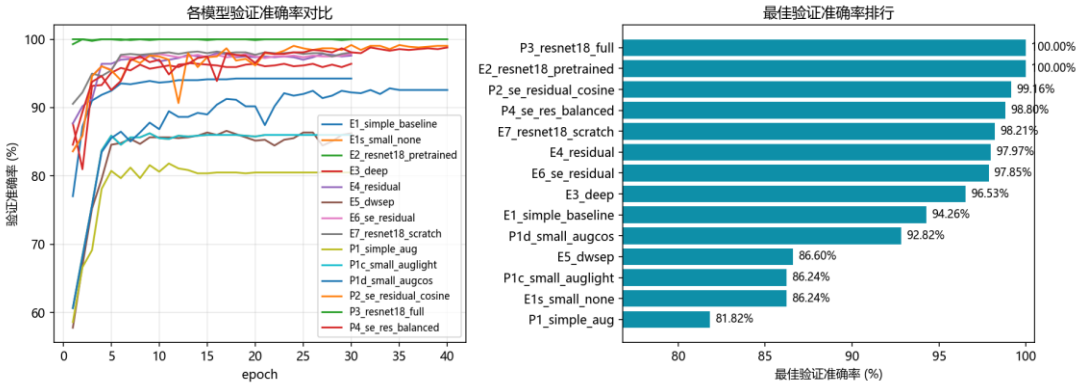


图 16　各实验验证准确率对比

三、实验结果

1.实验对比

表 3　各实验配置与验证准确率对比

实验	基础参数	结果（准确率）
基础实验-自定义模型	SimpleCNN（3 组卷积池化 + 两层全连接），输入 224×224，Adam lr=1e-4，batch=32，30 轮，StepLR 每 5 轮 ×0.1	94.26%
基础实验-调用的模型	torchvision ResNet18（ImageNet 预训练，替换 fc 为 7 类），其余参数同上	100.00%
创新实验-优化方法 1（改进网络模型）	把基线网络换成手写残差连接网络 ResidualCNN（两个 3×3 卷积 + shortcut，3 个 stage），其余参数与基线完全相同	97.97%
创新实验-优化方法 2（优化训练超参数）	在 SE-Residual 网络上同时改变调度与训练预算（30 轮 StepLR → 40 轮余弦退火），不改损失、不加标签平滑；对照为同网络 30 轮 StepLR 97.85%	99.16%

2. 实验结果

综上，两条创新均取得可复现的正向收益：残差结构与全局平均池化把参数量由 25.8M 降到 2.8M，同时把验证准确率从 94.26% 提升到 97.97%（+3.71 个百分点）；余弦退火（调度与训练预算同时改变：StepLR 30 轮 → 余弦 40 轮）把同一 SE-Residual 网络从 97.85% 提升到 99.16%，较对照约

+1.2 个百分点（best 对 best 为 11 张图，final 对 final 为 10 张图）。该值取自 best_val_acc，是在与报告同一份 836 张验证集上逐轮取的最大值，含约 1 张图的选择偏差（P2 best 99.16% / final 99.04%），收益方向可信，但不宜按 0.01 个百分点的精度解读。预训练 ResNet18 在 836 张验证集上第 1 轮即达 100.00%，此后各轮稳定在 99.88% 以上、末轮仍为 100.00%，说明该类间差异明显的叶片数据上迁移特征已足够可分。两处未取得正向收益的尝试各有成因：SE 通道注意力相对 ResidualCNN 未带来可测收益（97.97% → 97.85%，差 -0.12 个百分点，恰等于 836 张验证集中的 1 张图，落在方法分辨率之内，不可判方向）；强增强组 81.82% 并非增强有害，而是 StepLR 到第 30 轮把学习率降到 1e-10、训练集准确率仅 79.35% 造成欠拟合，补足训练预算后轻增强即回升到 92.82%。需要说明的是，上述结论基于单一种子（42）与单次数据划分，绝对数值存在约 ±1 个百分点的波动，跨数据集泛化仍需多划分重复实验验证。

为逐类检查模型表现，表 4 给出优化方法 2 的模型在 836 张验证集上的每类精确率、召回率与 F1；图 17 为每类各取 1 张验证样张的单张预测结果（真实→预测及置信度）。

表 4　各类别验证集指标（836 张，优化方法 2 模型）

类别	精确率(%)	召回率(%)	F1(%)	样本数
苹果黑星病	100.00	94.44	97.14	90
苹果黑腐病	100.00	100.00	100.00	89
苹果雪松锈病	95.12	97.50	96.30	40
苹果健康	98.34	100.00	99.16	237
葡萄黑腐病	100.00	99.41	99.70	169
葡萄健康	98.36	100.00	99.17	60
葡萄褐斑病	100.00	100.00	100.00	151



葡萄黑腐病→葡萄黑腐病　葡萄健康→葡萄健康　葡萄褐斑病→葡萄褐斑病

99.98%　99.95%　100.00%

图 17　七类验证集样例的单张预测（真实→预测，置信度）



图 18　无标签测试图推理示例（test/3.jpg，OpenCV 范式，置信度 99.81%）

错误分析与 Grad-CAM

本图汇总验证集全部 7 个错例的原始图像与解释图（上排原图，下排 Grad-CAM 或遮挡敏感度叠加）。其中 5 个是病害判成健康（苹果黑星病 3 个、黑腐病 1 个、雪松锈病 1 个），2 个为病种间混淆（苹果黑星病→雪松锈病 2 个）；置信度最高的错例为 91.2%（苹果黑星病→健康）。从 cam_check.json 看，5 张 Grad-CAM 的叶片内热区占比为 0.54–0.97，热区落在叶片上、没有糊在背景，但整体呈块状弥散，只能说明决策依据来自叶片的纹理与色泽，不能说明具体病斑位置。另有 2 个样本注意力落在背景，已改用遮挡敏感度图兜底（32×32 网格，分辨率粗，只能看大方向）。

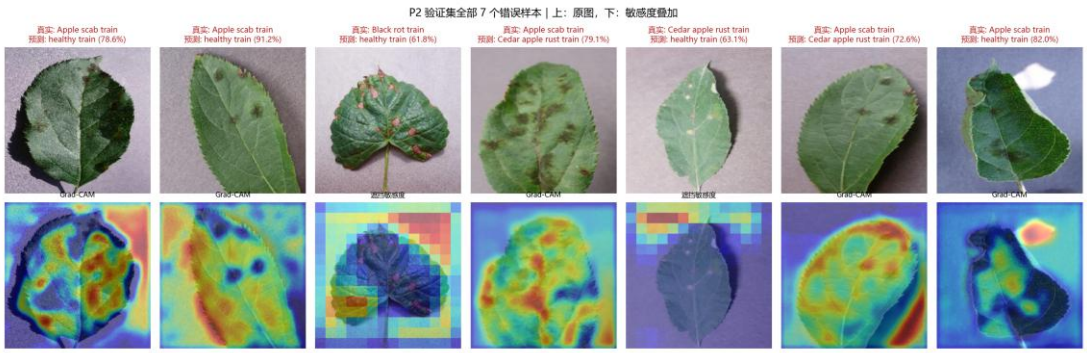


图 19　验证集 7 个错例与 Grad-CAM / 遮挡敏感度对照

需要说明的局限：这里只有 7 个错例，不做百分比或类别级结论；样本全部取自 836 张验证集，不外推到 1213 张无标签 test；Grad-CAM 与遮挡敏感度均为事后解释，只反映模型在该张图上的关注区域，不构成因果证据。

为检验上述结论是否只在自建验证集上成立，另用数据集自带的 val/ 子集做一次独立复核：这 466 张图带标签，但在全部实验中既未参与训练、也未参与模型选择（早停与 best_epoch 只看 836 张验证集），可视为干净的 holdout。8 个 run 的复核结果见表 5。优化方法 2（P2，余弦退火）在 holdout 上为 99.79%（错 1 张），与其验证集 99.16% 相差 0.63 个百分点，处于单次划分的噪声范围内，说明余弦退火带来的提升可复现而非划分偶然；优化方法 1 与 P4 在 holdout 上同样保持正收益（99.14% 与 99.36%）。相比之下，P3（ResNet18 全程微调）从验证集 100.00% 回落到 99.36%（错 3 张），说明此前的 100.00% 是 836 张验证集饱和叠加 best_epoch 选取偏置的结果，并非泛化上限。E2（ResNet18 预训练）在独立 holdout 上仍为 100.00%（466 张中错 0 张），说明饱和主要发生在训练期验证集；P3 全程微调后回落 0.64 个百分点，提示更强的拟合带来轻微过拟合，但两者仅相差 3 张图，属噪声量级，不据此判断优劣。需要说明的是，holdout 仅 466 张，错 1 张即 0.21 个百分点；其中最小的苹果雪松锈病类只有 22 张，错 1 张就是 4.55 个百分点。因此表 5 只用于方向性复核（结论是否在未参与选择的数据上依然成立），不宜按小数位做精确排名。

表 5　独立 holdout（466 张）复核结果

实验（run）	验证集 836 张 (%)	独立 holdout 466 张 (%)	错分张数
E1_simple_baseline	94.26	96.78	15
E2_resnet18_pretrained	100.00	100.00	0
E3_deep	96.53	98.07	9
E4_residual	97.97	99.14	4
E6_se_residual	97.85	98.50	7
P2_se_residual_cosine	99.16	99.79	1
P3_resnet18_full	100.00	99.36	3
P4_se_res_balanced	98.80	99.36	3

表 6　单因素消融汇总

组名	模型	增强	调度·轮数	数据比例	val 836	holdout 466
E1_simple_baseline	simple (25.79M)	none	step 30	100%	94.26	96.78
E3_deep	deep (0.56M)	none	step 30	100%	96.53	98.07
E5_dwsep	dwsep	none	step 30	100%	86.60	未做
E4_residual	residual (2.78M)	none	step 30	100%	97.97	99.14
E6_se_residual	se_residual (2.80M)	none	step 30	100%	97.85	98.50
E7_resnet18_scratch	resnet18	none	step 30	100%	98.21	未做
P2_se_residual_cosine	se_residual (2.80M)	none	cosine 40	100%	99.16	99.79
P1_simple_aug	simple	strong	step 30	100%	81.82	未做
P1c_small_auglight	simple	light	step 30	100%	86.24	未做
P1d_small_augcos	simple	light	cosine 40	30%	92.82	未做
P5_simple_strong_cos40	simple	strong	cosine 40	100%	93.30	未做
E1s_small_none	simple	none	step 30	30%	89.00	未做
P6_small_none_cos40	simple	none	cosine 40	30%	90.55	未做

分辨率说明：验证集 836 张中错 1 张即 0.12 个百分点，holdout 466 张中错 1 张即 0.21 个百分点，holdout 中最小的苹果雪松锈病类只有 22 张、错 1 张即 4.55 个百分点。因此表中只有 SE 模块这一项（97.97% → 97.85%，-0.12 个百分点，恰为 1 张图）落在分辨率之内、方向不可判；其余差值均不少于 3 张图，方向可判：+1.31（调度与训练预算，含约 1 张图选择偏差）、+1.55（训练预算）、+2.27（轻增强）、-0.96（强增强+预算）、-5.26（数据量降至 30%）、-12.44（强增强+短预算，最强证据）。

评阅成绩：

成绩	
评语	